LoRA: Low-Rank Adaptation of Large Language Models

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang (Microsoft, 2021)

Abstract

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to specific tasks. Full fine-tuning of all parameters becomes prohibitively expensive as model sizes scale to hundreds of billions of parameters. We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks.

1. Introduction

Many applications in natural language processing rely on adapting one large, pre-trained language model to multiple downstream tasks. The primary drawback for full fine-tuning is that for each downstream task, we learn a completely new set of model parameters. When deploying 175B GPT-3 models, storing independent fine-tuned checkpoints is cost-prohibitive.

Methodology and Low-Rank Decomposition

Section 3: Problem Formulation and Weight Updates

3.1 Low-Rank Parameter Decomposition

A pre-trained weight matrix W_0 in R^{d x k} has an intrinsic rank that is much lower than its ambient dimensions during task adaptation. For W_0, we constrain its update by representing the update with a low-rank decomposition: Delta W = B * A, where B is in R^{d x r} and A is in R^{r x k}, with the rank r much smaller than min(d, k). During training, W_0 is frozen and receives no gradient updates, while A and B contain trainable parameters. We use a random Gaussian initialization for A and zero for B, so Delta W = B A is initially zero at the beginning of training.

3.2 Scaling Factor alpha

When multiplying by input x, the forward pass yields the modified activation formula:

$$h = W_0 x + \Delta W x = W_0 x + (\alpha / r) B A x$$

Results and Constraints

Section 5: Empirical Benchmarks and Limitations

5.1 Memory and Parameter Efficiency Table

Empirical comparison of trainable parameter counts and memory usage during adaptation of GPT-3 175B:

Method	Trainable Params	GPU Memory	Checkpoint Size
Full Fine-Tune	175,000M (100%)	1.2 TB	350 GB
Prefix Tuning	350M (0.2%)	450 GB	700 MB
LoRA (r=4)	18M (0.01%)	350 GB	35 MB
LoRA (r=8)	36M (0.02%)	360 GB	70 MB

6. Reported Limitations

LoRA introduces practical inference latency when switching between multiple task-specific adapters simultaneously in high-throughput multi-tenant serving environments if adapters cannot be dynamically fused into the base weights in real time. Processing batched inputs for diverse tasks in a single forward pass remains non-trivial.